

11 – Introduction to NoSQL & MongoDB Fundamentals

DATABASE SYSTEM

Vincentius Kurniawan S.Kom., M.Eng.Sc.,

COURSE OBJECTIVES

GOALS.

IF-351 – 2025

- 1 Explain and Define what is [NoSQL](#)
- 2 Explain the considerations and better approach between different [types of databases](#) in NoSQL
- 3 Explain [MongoDB](#) and Implement their basic commands
- 4 Explain the MongoDB Architecture including [documents](#) and [collections](#).



OUTLINES.

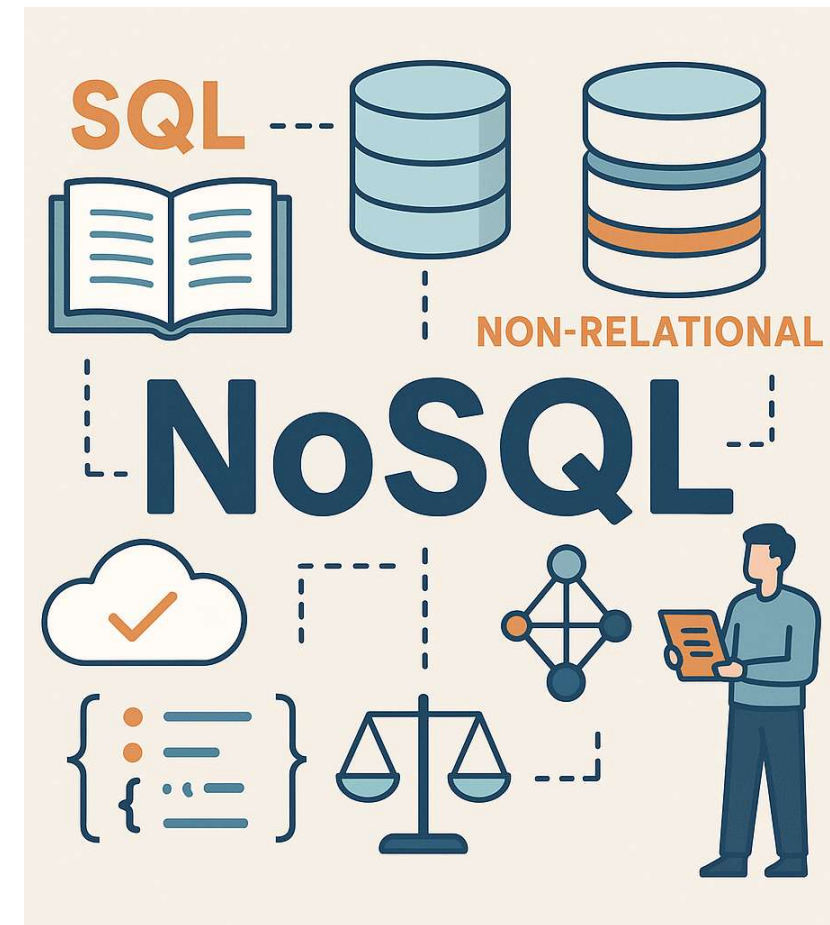
- Why NoSQL
- Types of NoSQL Databases
- MongoDB Overview
- Basic MongoDB Commands
- Documents

WHAT IS NoSQL?

The term “NoSQL database” is commonly used to describe any [non-relational database](#). While some interpret NoSQL as meaning "non-SQL," others believe it stands for "not only SQL," reflecting its ability to incorporate both structured and unstructured data.

Regardless of interpretation, NoSQL databases are known for [storing data in a flexible and intuitive manner](#). Unlike SQL, which is specifically a query language for interacting with databases, NoSQL refers to a [broader approach to database management](#) that allows for diverse data models and scalable architectures.

“Where it all begins”



WHY NoSQL?

Two primary reasons why people consider using a NoSQL database.

1

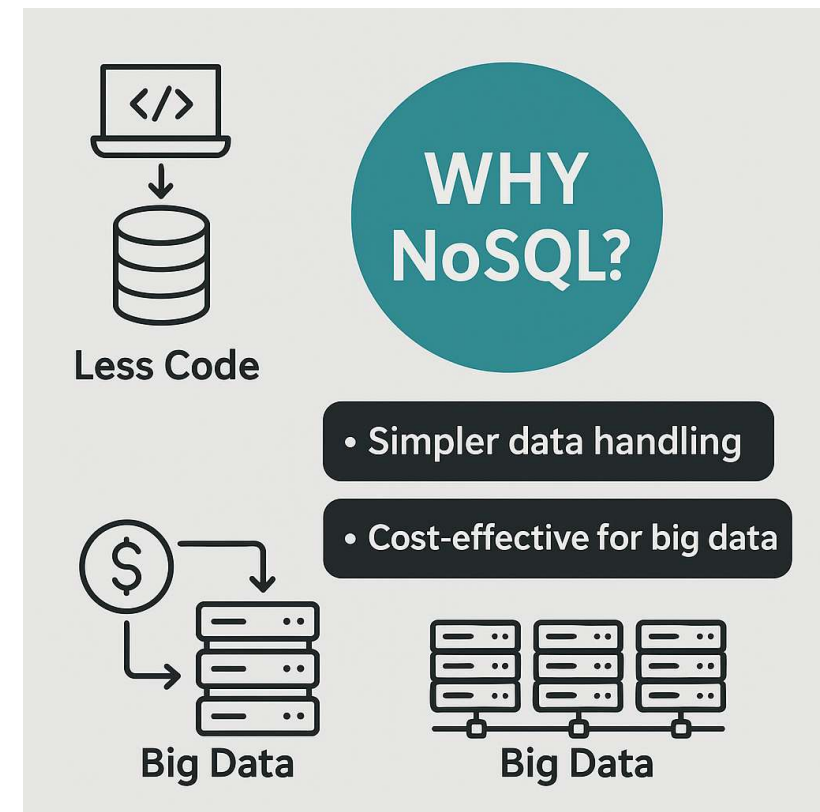
Application development productivity.

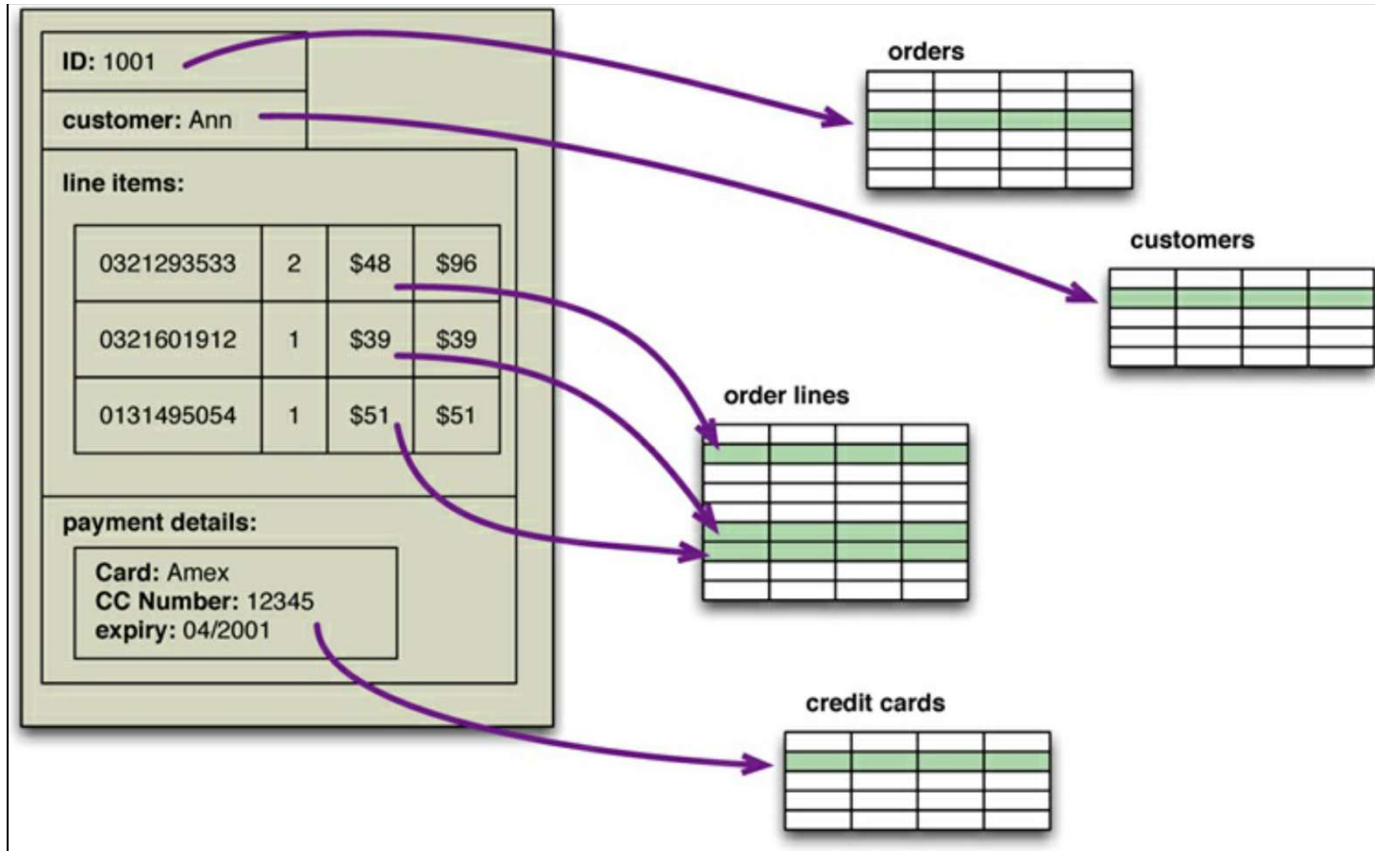
Many applications spend a lot of time and effort converting data between their internal memory and a relational database. NoSQL databases offer a more [flexible data structure](#) that can better match an application's needs, making this process simpler and reducing the amount of code required for development, debugging, and future updates.

2

Large-scale data.

Organizations are increasingly capturing and processing large amounts of data at faster speeds, but relational databases often struggle to keep up due to their reliance on a single machine, making them costly and inefficient for scaling. [NoSQL databases, designed to run on clusters of smaller](#), more affordable machines, provide a better solution for managing big data efficiently.





INTRODUCTION TO NO-SQL

WHY NoSQL? (CONT.D)

For application developers, the biggest frustration has been what's commonly called the **impedance mismatch**: the difference between the relational model and the in-memory data structures. The relational data model organizes data into a structure of tables and rows, or more properly, relations and tuples.

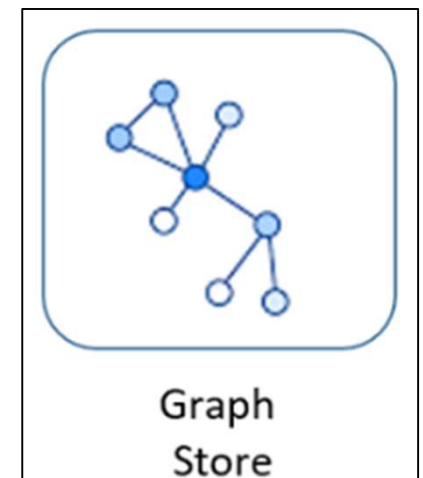
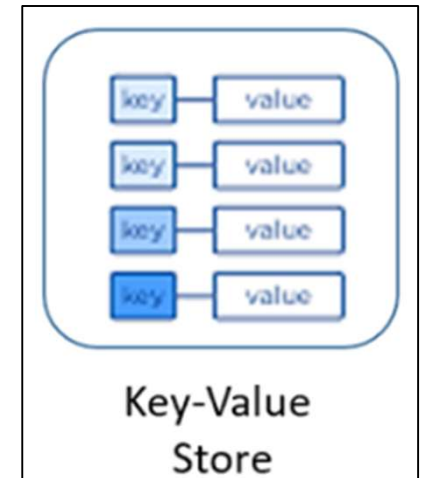
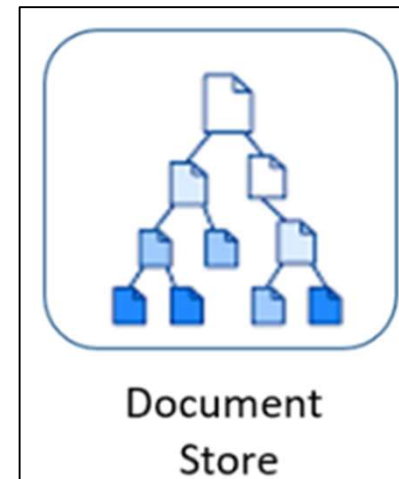
This foundation on relations provides a certain elegance and simplicity, but it also introduces limitations. In particular, the values in a relational tuple have to be simple—they cannot contain any structure, such as a **nested record** or a **list**. If you want to use a richer in memory data structure, you have to translate it to a relational representation to store it on disk

TYPES OF NOSQL DATABASES

NoSQL databases are a different kind of database that don't use the traditional row-and-column structure like SQL databases. Instead, they store data in different ways depending on the type.

There are four main types of NoSQL databases:

- Document databases
- Key-value stores
- Column-oriented databases
- Graph databases



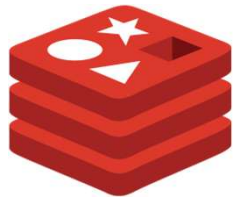
TYPES OF NOSQL DATABASES

DOCUMENT DATABASES

Document databases store data in formats like JSON, BSON (Binary JSON) or XML, allowing easy indexing and retrieval without complex conversions. Their flexible structure helps developers modify data without needing database administrators, speeding up development. They scale efficiently for large data and high-traffic applications like eCommerce and mobile apps

Example: MongoDB, CouchDB, Cloud Firestore, Amazon DocumentDB





redis



MEMCACHED



RocksDB

HAZELCAST™



Valkey

TYPES OF NOSQL DATABASES

KEY-VALUES STORES

This is [the simplest type of NoSQL database](#). Every element is stored as a [key-value pair consisting of an attribute name \("key"\) and a value](#). This database is like an RDBMS with two columns: the attribute name (such as "city") and the value (such as "Jakarta").

Use cases for NoSQL databases include shopping carts, user preferences, and user profiles

Example: Redis, MemCache, RocksDB, HazelCAST, Valkey,

TYPES OF NOSQL DATABASES

GRAPH DATABASES

Graph databases specialize in storing and analyzing relationships between data elements, [using nodes to represent objects and links to define their connections](#). Unlike SQL databases that require complex joins, graph databases provide direct and efficient relationship mapping, making them useful for applications like [fraud detection](#) and [social networks](#).

[They often work alongside traditional databases](#) and can store data in different formats, such as structured documents (MongoDB), key-value pairs (Redis, Riak KV), or graph-based structures (Neo4j). Unlike relational databases, they are schema-agnostic, allowing flexible storage of both structured and semi-structured data.

Example: Neo4J, Amazon Neptune, JanusGraph





TYPES OF NOSQL DATABASES

COLUMN-ORIENTED

Column-oriented databases store data in columns instead of rows, [allowing faster analytics](#) by reading only relevant data without unnecessary memory usage. They offer efficient compression and quick aggregation, making them ideal [for analytical tasks](#) like calculating yearly sales.

However, their writing process requires multiple disk write events, [making consistency more challenging](#) compared to relational databases, which store row data contiguously for smoother writing.

Example: **Apache Cassandra**, **Apache HBASE**, **Druid**, **SycllaDB**

TYPES OF NOSQL DATABASES (LLM'S HIGHLIGHT)

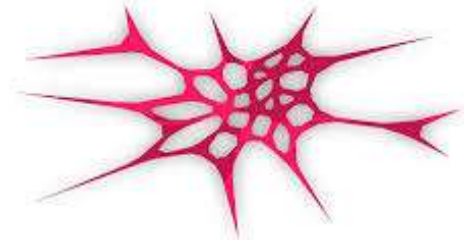
VECTOR DATABASES

Vector databases are specialized databases designed to store and search high-dimensional vectors, which represent data in numerical form. They are [commonly used in AI applications](#) like large language models (LLMs) because they enable fast similarity searches, making it easier to find relevant information based on patterns rather than exact matches.

Instead of storing traditional structured data like rows and columns, vector databases store data as mathematical vectors. [Each vector represents an object, such as text, images, or audio, in a multi-dimensional space.](#) When searching for similar objects, the database calculates the distance between vectors using methods like [cosine similarity](#) or [Euclidean distance](#).

Example: FAISS (by Facebook), Pinecone, Weaviate

FAISS
Scalable Search With Facebook AI



 **Pinecone**

 **Weaviate**

WHEN (NOT) TO USE NOSQL ?

Scenario	When to Use NoSQL	When to Use Relational DB (SQL)
Handling large-scale unstructured data	Its ideal for managing unstructured or semi-structured data, such as JSON files or multimedia.	Its struggle with unstructured data due to rigid schemas.
Dynamic / evolving data models	Use It when your data model changes frequently or requires flexible schema definitions.	It better when the data model is stable and well-defined from the outset.
Real-time analytics / High-velocity data	It is suited for real-time processing in applications like IoT, social media, or e-commerce analytics.	It may lag in high-velocity scenarios due to their strict ACID compliance.
Complex relationships & joins	Avoid NoSQL if your application relies heavily on joins or complex relationships; graph databases are an exception.	It excel at managing complex relationships with structured data and foreign keys.
Horizonal scalability requirements	It designed for horizontal scaling across multiple servers, making them ideal for distributed systems.	It typically scale vertically, which can become expensive and less efficient.
String transactional integrity (ACID)	It often trade strict consistency for scalability, suitable for eventual consistency models.	Use relational databases for applications like banking or financial systems that require ACID properties.

WHEN (NOT) TO USE NOSQL ? (CONT.D)

Scenario	When to Use NoSQL	When to Use Relational DB (SQL)
Support for multi-model use cases	NoSQL databases offer multi-model support, such as combining document, key-value, and graph models.	Relational databases are less flexible for multi-model use cases.
Data archival / simple read operations	key-value stores are better for caching or systems with high-read, low-write requirements.	Its overkill for simple key-value or archival needs.
Applications requiring complex querying	It may fall short due to limited or specialized query capabilities.	It is ideal for complex queries requiring SQL, joins, and aggregations.
Short development cycles	NoSQL supports rapid development and iteration due to its flexible schema and scalability.	Relational databases require upfront schema design, slowing development in dynamic environments.
System handling structured, tabular data	Not ideal unless specific use cases demand alternative models.	Relational databases are the best choice for structured, tabular data with predictable relationships.

NoSQL

Relational Databases (SQL)

Data Structure	Schema-less accommodates a variety of data types.	Structured, tabular schema, fixed data types.
Scalability	Designed for horizontal scaling (more servers).	Typically scaled vertically (more CPU, RAM, SSD)
Flexibility	High flexibility	Limited flexibility
ACID Properties	Some sacrifice ACID compliance for performance and scalability, others offer tunable consistency.	Adhere to ACID properties for reliable transactions.
Query Language	Often uses simpler query languages, may provide APIs for data manipulation.	Uses SQL for defining and manipulating data.
Relations	Some can handle relations effectively (like graph databases), but not all are designed to do so.	Can handle relations between different data points effectively.
Use Cases	Used for large data sets, real-time applications, and high write volume systems: social media , big data applications .	Ideal for applications requiring complex queries, and multi-row transactions: financial systems , e-commerce applications .

COMPARISON

RELATIONAL VS NOSQL DATABASES

MONGODB

MongoDB is a [document-oriented database](#), not a relational one. A document-oriented database replaces the concept of a “row” with a more flexible model, the “document.”

By allowing [embedded documents](#) and [arrays](#), the document oriented approach makes it possible to represent complex hierarchical relationships with a single record.

There are also [no predefined schemas](#): a [document’s keys and values are not of fixed types or sizes](#). Without a fixed schema, adding or removing fields as needed becomes easier. Generally, this makes development faster as developers can quickly iterate.



MONGODB FEATURES

MongoDB is a general-purpose database, so aside from creating, reading, updating, and deleting data, it provides most of the features you would expect from a database management system and many others that set it apart. These include:

1 Indexing

It supports various indexing types, including secondary indexes on [nested documents](#) and [arrays](#), allowing flexible data modeling

2 Aggregation

The framework uses data [processing pipelines](#) to handle analytics by breaking tasks into simple stages.

3 Special collection and index types

It offers TTL collections for expiring data like sessions, [capped collections](#) for storing recent data like [logs](#), and partial indexes to improve efficiency and save storage space.

4 File storage

It supports an easy-to-use protocol for [storing large files](#) and file [metadata](#).

DOCUMENTS

A **document** is an ordered set of **keys** with associated **values**. Every document has a special key, "`_id`", that is unique within a collection. Most documents will be more complex than this simple one and often will contain multiple key/value pairs.

Keys

- The keys in a document are strings. Any UTF-8 character is allowed in a key
- Keys must not contain the character `\0` (the null character). This character is used to signify the end of a key.
- The `.` and `$` characters have some special properties and should be used only in certain circumstances

A terminal window titled "Documents - MongoDB" with three colored window control buttons (red, yellow, green) on the left. It displays a JSON document with four lines of code: a left brace, a key-value pair "greeting" with value "Hello, world!", another key-value pair "views" with value 3, and a right brace. The keys are highlighted in green, the string value is in yellow, and the integer value is in blue.

```
1 {  
2   "greeting" : "Hello, world!",  
3   "views" : 3  
4 }
```

DOCUMENTS (CONT.D)

Values in documents are not just “blobs.” They can be one of several different data types (or even an entire [embedded document](#))

MongoDB is [type-sensitive](#) and [case-sensitive](#). For example, these documents are distinct:

Documents - Case and Type Sensitive

```
1 {"count" : 5}
2 {"count" : "5"}
3 {"Count" : 5}
```

Documents in MongoDB **cannot** contain duplicate keys.

Documents - Not a Legal Document

```
1 {
2   "greeting" : "Hello, world!",
3   "greeting" : "Hello, MongoDB!"
4 }
```

COLLECTIONS

A **collection** is a group of documents. If a document is the MongoDB analog of a row in a relational database, then a collection can be thought of as the analog to a **table**

Collections have **dynamic schemas**. This means that the documents within a single collection can have any number of different “shapes.” For example, both of the following documents could be stored in a single collection:

```
Collection - Dynamic Schemas

1 {"greeting" : "Hello, world!", "views": 3}
2 {"signoff": "Good night, and good luck"}
```

Things to discuss:

Why do we need separate collection at all? With no need for separate schemas for different kinds of documents, why should we use more than one collection?

DATABASES

In addition to grouping documents by collection, [MongoDB groups collections](#) into databases. A single instance of MongoDB can host several databases, each grouping together zero or more collections. A good rule of thumb is to store all data for a single application in the same database.

Database and Collection naming rules:

- The empty string (``) is not a valid database name.
- A database name cannot contain any of these characters: `/`, `\`, `.`, `"`, `*`, `<`, `>`, `:`, `|`, `?`, `$`, (a single space), or `\0` (the null character). Basically, stick with alphanumeric ASCII.
- Database names [are case-insensitive](#).
- Database names are limited to a [maximum of 64 bytes](#).

Notes:

In MongoDB, a newly created database won't appear in the results of the `show dbs` command until it contains at least one collection with data.

DATA TYPES IN MONGODB

MongoDB supports a wide range of data types as values in documents

- Null
- Boolean
- Number
- String
- Date
- Regular Expression
- Array
- Embedded document
- Object ID
- Binary Data
- Code

DATA TYPES IN MONGODB (CONT.D)

Array

Sets or lists of values can be represented as arrays:

```
{"x" : ["a", "b", "c"]}
```

Regular Expression

Queries can use regular expressions using JavaScript's regular expression syntax:

```
//Regular Expression  
{"x" : /foobar/i}
```

Object ID

An object ID is a 12-byte ID for documents:

```
{"x" : ObjectId( )}
```

Embedded document

Documents can contain entire documents embedded as values in a parent document:

```
{"x" : {"foo" : "bar"}}
```

DATA TYPES IN MONGODB (CONT.D)

There are also a few less common types that you may need, including:

Binary data

Binary data is a string of arbitrary bytes. It cannot be manipulated from the shell.

Binary data is the only way to save non-UTF-8 strings to the database.

Code

MongoDB also makes it possible to store arbitrary JavaScript in queries and documents:

```
{"x" : function() { /* ... */ }}
```


GETTING STARTED

MONGODB SHELL

Go to the [MongoDB Community Server download page](https://www.mongodb.com/try/download/shell) and choose following your PC specification

If you start the shell with `--nodb`, it will start up without attempting to connect to anything:

```
λ mongosh --nodb
Current Mongosh Log ID: 6858d950632148d5c36c4bcf
Using Mongosh: 2.5.1
mongosh 2.5.3 is available for download: https://www.mongodb.com/try/download/shell
For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
```

Once started, you can connect to a mongod at your leisure by running `new Mongo("hostname")` :

```
> conn = new Mongo("localhost:27017")
mongodb://localhost:27017/?directConnection=true&serverSelectionTimeoutMS=2000
> db = conn.getDB("myDB")
myDB
myDB> console.log("hello world")
hello world

myDB> x=5+2
7
myDB>
```

GETTING STARTED

MONGODB SHELL (CONT.D)

For MongoDB-specific functionality, the shell includes built-in help that can be accessed by typing `help`:

```
myDB> help

Shell Help:

log
  Write a custom info/warn/error/fatal/debug message to the log file
  'log.info(<msg>)': Write a custom info/warn/error/fatal/debug message to the log file
  'log.getPath()': Gets a path to the current log file

use
  Set current database

show
  'show databases'/'show dbs': Print a list of all available databases
  'show collections'/'show tables': Print a list of all collections for current database
  'show profile': Prints system.profile information
  'show users': Print a list of all users for current database
  'show roles': Print a list of all roles for current database
  'show log <type>': log for current connection, if type is not set uses 'global'
  'show logs': Print all logs

exit
  Quit the MongoDB shell with exit/exit().exit
quit
  Quit the MongoDB shell with quit/quit()
Mongo
  Create a new connection and return the Mongo object. Usage: new Mongo(URI, options [optional])
connect
  Create a new connection and return the Database object. Usage: connect(URI, username [optional], password [optional])
it
  result of the last line evaluated; use to further iterate
version
  Shell version
load
  Loads and runs a JavaScript file into the current shell environment
enableTelemetry
  Enables collection of anonymous usage data to improve the mongosh CLI
disableTelemetry
  Disables collection of anonymous usage data to improve the mongosh CLI
passwordPrompt
  Prompts the user for a password
sleep
  Sleep for the specified number of milliseconds
print
  Prints the contents of an object to the output
printjson
  Alias for print()
convertShardKeyToHashed
  Returns the hashed value for the input using the same hashing function as a hashed index.
cls
  Clears the screen like console.clear()
isInteractive
  Returns whether the shell will enter or has entered interactive mode

For more information on usage: https://docs.mongodb.com/manual/reference/method
myDB>
```

SUMMARY

Why NoSQL

NoSQL databases offer flexible schemas and horizontal scalability, making them ideal for handling large volumes of unstructured or semi-structured data. They emerged as a solution to the limitations of traditional relational databases in big data and real-time applications

Types of NoSQL Databases

There are four main types: document-based, key-value, column-family, and graph databases—each optimized for different data models and use cases like content management, caching, analytics, and relationship mapping.

MongoDB Overview

MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents within collections. It supports high performance, horizontal scalability, and dynamic schemas, making it popular for modern applications.

Basic MongoDB Commands

Common commands include `use` to switch databases, `db.createCollection()` to create collections

Documents

In MongoDB, documents are the basic unit of data, structured as key-value pairs and stored in BSON format. They can contain nested documents and arrays, allowing for rich and flexible data representation.

REFERENCES

Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7th edition, Addison Wesley

Van der Lans, R. F. (2007). SQL for MySQL developers. Addison Wesley.

Swan, M. (2015). Blockchain: Blueprint for a new economy. O'Reilly Media.

Bradshaw, D., Brazil, K., & Chodorow, K. (2019). MongoDB: The definitive guide—Powerful and scalable data storage. O'Reilly Media.

Copeland, R. (2013). MongoDB applied design patterns. O'Reilly Media.



NEXT OUTLINES

NEXT WEEK

- CRUD Operations
- Data Modeling in NoSQL



KEEP IN TOUCH

THANK YOU

vincentius.kurniawan@lecturer.umn.ac.id



@aixeta#2683

